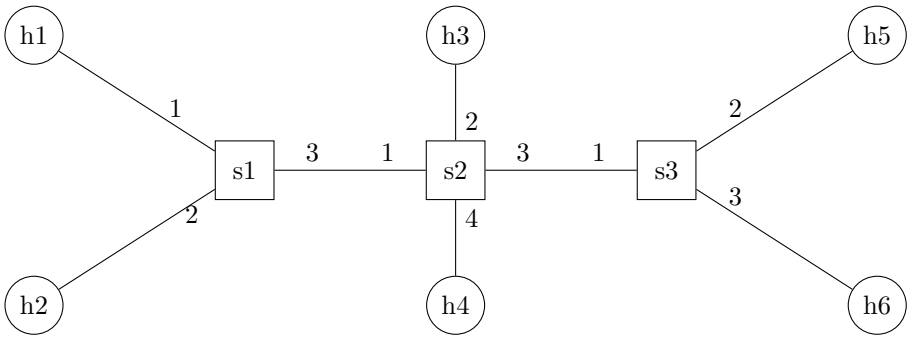
Piattaforme Software per la Rete (Mod. 1) – 06/07/2017

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura.



(a) Scrivere il contenuto delle tabelle openflow dopo l'accensione di tutti i nodi.

Soluzione:

s1

Priority	Match	Action
0	*	output controller

s2

Priority	Match	Action
0	*	output controller
1	eth_dst = MAC3 eth_type = IPv4 ip_proto = TCP	output 2

s3

Priority	Match	Action
0	*	output controller

L'host h1, porta 3456 apre una connessione verso h3, porta 123.

(b) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, s3.

Soluzione:

$s1 \rightarrow C$: packet_in(MAC1 > MAC3 IPv4 IP1 > IP3 TCP[SYN] 3456 > 123)
nota: viene inserita una regola in s1

$s1 \rightarrow s2$: MAC1 > MAC3 IPv4 IP1 > IP3 TCP[SYN] 3456 > 123

$s2 \rightarrow h3$: MAC1 > MAC3 IPv4 IP1 > IP3 TCP[SYN] 3456 > 123

(c) L'host h3 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, s3.

Soluzione:

$s2 \rightarrow C$:	packet_in(MAC3 > MAC1 IPv4 IP3 > IP1 TCP[SYN,ACK] 123 > 3456)
$s2 \rightarrow s1, s2, h4$:	MAC3 > MAC1 IPv4 IP3 > IP1 TCP[SYN,ACK] 123 > 3456
$s1 \rightarrow h1$:	MAC3 > MAC1 IPv4 IP3 > IP1 TCP[SYN,ACK] 123 > 3456
$s3 \rightarrow C$:	packet_in(MAC3 > MAC1 IPv4 IP3 > IP1 TCP[SYN,ACK] 123 > 3456)
$s3 \rightarrow h5$,$h6$:	MAC3 > MAC1 IPv4 IP3 > IP1 TCP[SYN,ACK] 123 > 3456

(d) L'host h1 riscontra la risposta. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, s3.

Soluzione:

$s1 \rightarrow C$:	packet_in(MAC1 > MAC3 IPv4 IP1 > IP3 TCP[SYN] 3456 > 123) nota: viene inserita una regola in s1, che sovrascrive la regola esistente
$s1 \rightarrow s2$:	MAC1 > MAC3 IPv4 IP1 > IP3 TCP[SYN] 3456 > 123
$s2 \rightarrow h3$:	MAC1 > MAC3 IPv4 IP1 > IP3 TCP[SYN] 3456 > 123

L'host h6, porta 7843 invia un datagramma UDP a h2, porta 543.

(e) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, 3.

Soluzione:

$s3 \rightarrow C$:	packet_in(MAC6 > MAC2 IPv4 IP6 > IP2 UDP 7843 > 543)
$s3 \rightarrow s2$:	MAC6 > MAC2 IPv4 IP6 > IP2 UDP 7843 > 544
$s2 \rightarrow C$:	packet_in(MAC6 > MAC2 IPv4 IP6 > IP2 UDP 7843 > 544)
$s2 \rightarrow s1$,$h3$,$h4$:	MAC6 > MAC2 IPv4 IP6 > IP2 UDP 7843 > 545
$s1 \rightarrow C$:	packet_in(MAC6 > MAC2 IPv4 IP6 > IP2 UDP 7843 > 545)
$s1 \rightarrow h1$,$h2$:	MAC6 > MAC2 IPv4 IP6 > IP2 UDP 7843 > 546

(f) Scrivere il contenuto finale delle tabelle openflow.

Soluzione:

s1

Priority	Match	Action
0	*	output controller
3	eth_dst = MAC1 eth_src = MAC 3 eth_type = IPv4 ip_proto = TCP	output 1

s2

Priority	Match	Action
0	*	output controller
1	eth_dst = MAC3 eth_type = IPv4 ip_proto = TCP	output 2

s3

Priority	Match	Action
0	*	output controller

Programma 1 (1/2) – Switch Features

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

    if (datapath.id == 2):
        match = parser.OFPMatch(eth_dst=MAC3,eth_type=IPv4,ip_proto=TCP)
        actions = [ parser.OFPActionOutput(2) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
```

Programma 1 (2/2) – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath
    ofproto = datapath.ofproto; parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp)
    pkt_tcp = pkt.get_protocol(tcp.tcp)
    data = msg.data

    if (pkt_tcp is not None) and (pkt_ipv4.src==IP1):
        match = parser.OFPMatch(
            eth_dst=pkt_eth.src,eth_src=pkt_eth.dst,eth_type=IPv4,ip_proto=TCP)
        actions = [ parser.OFPActionOutput(in_port) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
    elif (pkt_udp is not None):
        pkt_udp.dst_port = pkt_udp.dst_port+1
        pkt_udp.csum = 0 # azzera checksum esistente
        pkt.serialize()  # calcola nuovi checksum
        data = pkt.data

    if (datapath.id == 1) and (in_port < 3):
        outport = 3
    elif (datapath.id == 3) and (in_port > 1):
        outport = 1
    else:
        outport = ofproto.OFPP_FLOOD
    actions = [parser.OFPActionOutput(outport)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)
    datapath.send_msg(out)
```

Esercizio 2

Si consideri un sistema di misura del traffico basato su *consistent hashing*. In ogni nodo, ad ogni pacchetto è associato un *flow id* F tale che connessioni TCP distinte abbiano flow id distinti. Viene quindi calcolato $h(F)$, dove h è una funzione hash con output di 128 bit. Se $h(F)$ è maggiore di una soglia R, il flow id è inserito in un filtro di Bloom costituito da un array di $m = 32$ bit e $k = 3$ funzioni hash, g_1, g_2, g_3.

Ad uno di questi nodi arrivano pacchetti a velocità $C = 1\,\text{Gbit/s}$. La lunghezza media di una connessione TCP è $L = 10\,000$ byte.

(a) Calcolare la soglia R in modo da salvare nel filtro una connessione ogni 100.

Soluzione:

$$R = 2^{128} - \frac{2^{128}}{100}$$

(b) Calcolare la probabilità che il filtro di Bloom restituisca un falso positivo dopo avere inserito flow id per 1 ora.

Soluzione:

In un'ora si inseriscono $n = 3600 \cdot 10^9/80\,000 = 45 \cdot 10^6$ flussi.

La probabilità di falso positivo è:

$$p = (1 - \exp(-kn/m))^k \simeq 1$$

con $k = 3$ e $m = 32$.

(c) Dire se è corretto usare come funzioni hash le seguenti: per $g_1(x)$ gli ultimi 32 bit di $h(x)$, per $g_2(x)$ i penultimi ultimi 32 bit di $h(x)$, per $g_3(x)$ i terzultimi 32 bit di $h(x)$.

Soluzione:

L'array di Bloom è di 32 bit, quindi servono funzioni hash con output di 5 bit.

Esercizio 3

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Transport Protocol
48.0.0.0/4	*	*	80	TCP
38.0.0.0/7	51.0.0.0/8	80	22	TCP
*	48.0.0.0/4	*	53	*
38.0.0.0/8	*	*	123	UDP
51.0.0.0/8	38.0.0.0/8	23	*	UDP
*	38.0.0.0/7	25	80	*

(a) Definire il bitvector set per ciascun campo.

Soluzione:

Source Address	
38.0.0.0/7	011001
38.0.0.0/8	011101
48.0.0.0/4	101001
51.0.0.0/8	101011
*	001001

Destination Address	
38.0.0.0/7	100101
38.0.0.0/8	100111
48.0.0.0/4	101100
51.0.0.0/8	111100
*	100100

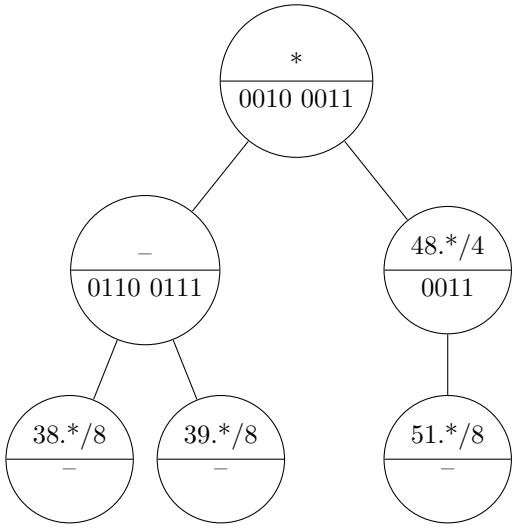
Source Port	
23	101110
25	101101
80	111100
*	101100

Dest. Port	
22	010010
53	001010
80	100011
123	000110
*	000010

protocol	
TCP	001111
UDP	111001
*	001001

(b) L'algoritmo di ricerca del prefisso IP sorgente (prima colonna) è implementato usando un multibit trie con passo 4. Disegnare il trie.

Soluzione:



(c) Si considerino il pacchetto (39.10.0.1, 39.0.0.2, 25, 80, TCP). Elencare le maschere identificate per ogni campo e identificare gli eventuali match.

Soluzione: Il pacchetto soddisfa la regola 6.

Esercizio 4

Si consideri un'interfaccia gestita da uno scheduler Deficit Round Robin. La velocità di trasmissione è 8 Gbit/s. Al tempo $t = 0$ sono presenti i seguenti pacchetti, di cui viene indicata la lunghezza in byte, ordinati dal più recente al più vecchio:

Coda 1: 50, 150, 250

Coda 2: 225, 50, 50

Coda 3: 100, 100, 100

Coda 4: –

Le code 1 e 2 hanno quanti di lunghezza 200. Le code 3 e 4 hanno quanti di lunghezza 100.

(a) Simulare l'algoritmo DRR, indicando il tempo al quale termina ogni trasmissione ed il deficit residuo.

Soluzione:

Round	Coda	Deficit iniziale	Deficit finale	Pacchetto trasmesso	Tempo
1	1	0	200	0	0
1	2	0	100	50 + 50	100
1	3	0	0	100	200
1	4	0	0	–	–
2	1	200	0	250+150	600
2	2	100	0	225	825
2	3	0	0	100	925
2	4	0	0	–	–
3	1	0	0	50	975
3	2	0	0	–	–
3	3	0	0	100	1075
3	4	0	0	–	–

(b) Ripetere la simulazione considerando che al tempo $t = 400\,\text{ns}$ si presenta alla coda 4 un pacchetto di lunghezza 100.

Soluzione:

Round	Coda	Deficit iniziale	Deficit finale	Pacchetto trasmesso	Tempo
1	1	0	200	0	0
1	2	0	100	50+50	100
1	3	0	0	100	200
1	4	0	0	–	–
2	1	200	0	250+150	600
2	2	100	0	225	825
2	3	0	0	100	925
2	4	0	0	100	1025
3	1	0	0	50	1075
3	2	0	0	–	–
3	3	0	0	100	1175
3	4	0	0	–	–